

Week 7: Benchmarking Python Against the Analytics Service Requirements

Aaron Foster

Southern New Hampshire University

CS 660: Advanced Programming Language Theory

Prof. Shadha Tabatabai

Aug 23rd, 2026

1. Executive Summary

Three sorting algorithms were implemented in pure Python, measured with a purpose-built benchmarking harness, and judged against the three requirements the stakeholders set. The short answer is that Python meets two of the three requirements outright, and meets the third only on fast hardware.

Requirement	Target	Measured	Verdict
R1 — Sort 100,000 integers	< 400 ms	84 ms on performance cores; 405 ms on efficiency cores	CONDITIONAL
R2 — Peak memory during sort	< 50 MB	3.8–5.5 MB of data; 21.6 MB process total	MET
R3 — Reusable benchmark harness	Provided	benchmark.py, 5 of 5 capabilities	MET

The finding that matters: requirement R1 is not a property of Python alone — it is a property of Python plus the machine. On the Apple M4 Max performance cores used for this study, the fastest pure-Python sort finished in 84 ms, comfortably inside the budget with $4.8\times$ headroom. Pinned to the same machine's efficiency cores, which stand in for the more modest hardware the phrase "standard hardware" implies, the same code averaged 405 ms and exceeded the 400 ms budget on **6 of 12 individual runs**. A requirement that a build passes half the time is not a requirement that is met.

The mitigation is equally clear. CPython's built-in `list.sort` — the same Timsort algorithm, implemented in C — finished the identical workload in 8 ms on performance cores and 22 ms on efficiency cores, passing the budget by more than an order of magnitude on both. Since the algorithms scale correctly (measured growth of roughly $2.1\times$ per doubling of n , exactly what $O(n \log n)$ predicts), the gap is not algorithmic. It is the cost of executing comparison logic in the interpreter rather than in compiled code.

Recommendation in one line: adopt Python for the analytics service, but treat pure-Python element-wise loops as a prototyping tool only, and require that any operation inside the 400 ms budget be delegated to compiled code.

2. Test Environment

Every number in this report came from the machine described below. Timing results are meaningless without it, so the harness records this block into its JSON output automatically.

Property	Value
Interpreter	CPython 3.13.0 (GIL enabled)
Operating system	macOS 26.5.1, arm64
Processor	Apple M4 Max — 14 cores (10 performance, 4 efficiency)
Memory	36 GB
Timer	time.perf_counter (monotonic, highest available resolution)
Memory instrument	tracemalloc, plus resource.getrusage for process RSS
Input	100,000 pseudorandom integers in $[0, 10^9]$, fixed seed 660
Iterations	5 timed runs per measurement, 1 untimed warm-up

A note on "standard hardware." An M4 Max is well above the hardware a typical deployment target offers, so measuring only on it would flatter Python and answer the wrong question. Apple silicon makes a controlled comparison unusually easy: `taskpolicy -b` runs a process at background quality-of-service, which the scheduler confines to the efficiency cores.

That yields two hardware tiers from one machine, with the operating system, interpreter, and input held identical — the only variable is core speed. Both tiers are reported throughout.

3. Deliverable 1 — Quick Sort, Merge Sort, and Heap Sort

All three algorithms are implemented in `sorts.py` in pure Python, with no calls to built-in sorting. They share one interface, `sort_function(data: list) -> list`, so the harness can accept any of them interchangeably.

Algorithm	Strategy	Time	Extra space	Stable	In place
<code>quick_sort</code>	Median-of-three pivot, Hoare partition	$O(n \log n)$ avg	$O(\log n)$	No	Yes
<code>merge_sort</code>	Top-down split, stable merge	$O(n \log n)$ all	$O(n)$	Yes	No
<code>heap_sort</code>	Bottom-up max-heap, sift-down	$O(n \log n)$ all	$O(1)$	No	Yes

3.1 Two deliberate hardening decisions in quick sort

A textbook quick sort fails outright at this input size, so two changes were necessary and both are worth stating because they change what the benchmark measures. To avoid quicksort's $O(n^2)$ worst-case performance on already-sorted arrays, the implementation uses median-of-three pivot selection (taking the median of the first, middle, and last elements), which Section 5.2 confirms turns sorted and reverse-sorted data into best-case scenarios that run faster than random input.

Additionally, the algorithm relies on tail-call elimination for the larger partition by looping on the larger side and recursing only on the smaller side, strictly bounding the stack depth at $O(\log n)$. Because CPython does not natively optimize tail calls and caps recursion at 1,000 frames, this optimization is a hard correctness constraint rather than a stylistic preference, ensuring the program doesn't crash when sorting large inputs.

3.2 Why `list.sort` appears in every table

CPython's built-in `list.sort` is benchmarked alongside the three implementations. It is not one of the algorithms the activity asks for; it is the *control*. It runs the same data on the same machine through the same harness, but executes as compiled C. Any difference between it and the pure-Python sorts isolates the cost of the interpreter from the cost of the algorithm, which is precisely the question section 6 has to answer.

4. Deliverable 2 — The Benchmarking Harness

`benchmark.py` is a standalone module that knows nothing about sorting. It accepts any callable and any input sequence, which is what makes it reusable rather than a script written once for this assignment. The table below maps each required capability to its implementation.

Required capability	Implementation
Accepts a sort function and an input list as parameters	<code>benchmark(sort_function, data, ...)</code> — validates both and raises on misuse
Runs the function a configurable number of iterations	<code>iterations</code> parameter (default 5), plus a separate <code>warmup</code> parameter
Measures average execution time with <code>time.perf_counter</code>	<code>_time_pass()</code> reports mean, min, max, and standard deviation
Captures peak memory usage with <code>tracemalloc</code>	<code>_memory_pass()</code> reports the peak allocated by the function itself
Outputs results to the console or saves them to a file	<code>print_table()</code> to console; <code>write_csv()</code> and <code>write_json()</code> to disk

4.1 Three measurement decisions that change the numbers

1. Timing and memory are collected in separate passes. tracemalloc intercepts every allocation CPython makes and inflates runtime several-fold. Timing code while it runs measures the profiler, not the program. The harness therefore runs the timing loop with tracing off, then performs one additional traced run purely for the memory figure.

2. Every iteration receives a fresh copy of the input, made outside the timed region. quick_sort and heap_sort mutate their argument. Without a fresh copy, the second iteration would be sorting already-sorted data — a completely different workload that would silently understate the average. The copy is made before the clock starts so its cost is not attributed to the sort.

3. Garbage collection is forced before each timed iteration, outside the timed region. A collection pass triggered by earlier work would otherwise land mid-measurement and appear as a phantom outlier.

4.2 A measurement bug worth reporting

The harness originally measured the input's memory footprint by copying an existing list under tracemalloc. That under-reported the input by a factor of five, and the reason is the very memory model this activity asks about: copying a list of integers allocates a new array of pointers, but *reuses the existing integer objects*, which account for most of the footprint. The corrected harness accepts a factory callable and builds the data under tracing, so the integer objects are counted.

footprint measured from a factory : 3.82 MB <- correct

footprint measured from a copy : 0.76 MB <- counts pointers only

The bug is reported rather than quietly fixed because the size of the error is itself the clearest single demonstration of how CPython represents a list of integers.

5. Results

5.1 Headline result: 100,000 random integers

Table 1 — Both hardware tiers, 400 ms budget, 100,000 random integers

Algorithm	P-core avg	E-core avg	Peak alloc	Data + alloc	R1 verdict
quick_sort	83.6 ms	405.3 ms	0.00 MB	3.82 MB	P pass / E fail
merge_sort	115.6 ms	517.2 ms	1.65 MB	5.47 MB	P pass / E fail
heap_sort	144.5 ms	667.6 ms	0.00 MB	3.82 MB	P pass / E fail
list.sort [C]	8.0 ms	22.1 ms	0.38 MB	4.20 MB	pass on both

Read across the last row first. The same machine, the same data, and the same 400 ms budget: compiled Timsort passes on the slow cores with a 18× margin, while every pure-Python

implementation fails there. Read down the peak-allocation column and the memory story is the opposite — the in-place sorts allocate *nothing at all*, and even merge sort's 1.65 MB is trivial against a 50 MB budget.

5.2 Stability of the E-core result

Because the E-core average for quick sort lands almost exactly on the budget, an average alone is misleading. Twelve individual iterations were recorded to characterize the spread.

Table 2 — Efficiency cores, 12 timed iterations, 100,000 random integers

Algorithm	Mean	Min	Max	Std dev	Iterations over 400 ms
quick_sort	405.3 ms	322.9 ms	583.3 ms	80.9 ms	6 of 12
merge_sort	517.2 ms	443.8 ms	760.2 ms	83.0 ms	12 of 12
heap_sort	667.6 ms	592.7 ms	857.5 ms	83.0 ms	12 of 12

Quick sort is a coin flip against the requirement. That is a worse outcome than a clean failure, because it is the kind of result that passes in development and fails intermittently in production.

5.3 Sensitivity to input distribution

Table 3 — Average milliseconds by input shape, performance cores, n = 100,000

Algorithm	random	sorted	reversed	nearly sorted	few unique
quick_sort	83.6	61.8	63.1	63.5	77.3
merge_sort	115.6	81.2	82.2	104.8	110.7
heap_sort	144.5	141.7	134.9	142.0	135.1
list.sort [C]	8.0	0.28	0.23	1.07	5.19

- Median-of-three works. Sorted and reverse-sorted input are quick sort's fastest cases (62–63 ms), not its worst. A naive pivot choice would have produced $O(n^2)$ behavior and never finished.
- Heap sort is indifferent to input order — 135 to 145 ms across every distribution. It always builds a heap and always extracts n times, so pre-existing order buys it nothing.
- list.sort collapses to 0.23 ms on sorted and reverse-sorted data, a $35\times$ speedup over its own random-input time. Timsort detects existing runs and merges them, an adaptive optimization none of the three textbook algorithms has. Real analytics input is frequently near-sorted, so this is a practical advantage, not a benchmark artifact.

5.4 Scaling behavior

Table 4 — Average milliseconds vs input size, performance cores, random input

n	quick_sort	merge_sort	heap_sort	list.sort [C]
10,000	7.02	9.85	11.12	0.61
25,000	19.52	26.11	31.38	1.78
50,000	39.61	54.67	65.98	3.85
100,000	84.61	117.60	145.42	8.35
200,000	176.01	246.22	337.40	18.36

Measured growth per doubling of n — $O(n \log n)$ predicts roughly $2.1\times$:

quick_sort 2.78x 2.03x 2.14x 2.08x

merge_sort 2.65x 2.09x 2.15x 2.09x

heap_sort 2.82x 2.10x 2.20x 2.32x

list.sort [C] 2.90x 2.16x 2.17x 2.20x

This is the single most important diagnostic in the report. Every implementation, interpreted and compiled alike, tracks the $O(n \log n)$ prediction. Nothing degrades toward $O(n^2)$, and the pure-Python sorts do not scale *worse* than the C one — they are simply displaced upward by a constant factor of roughly ten. The algorithms are correct; the per-operation cost is the

problem. That distinction determines which mitigations can work, because no amount of algorithmic tuning removes a constant factor of ten.

Heap sort's final ratio of $2.32\times$ is the one mild exception, and it is a cache effect: sift-down jumps between indices i , $2i+1$, and $2i+2$, so as the array outgrows cache its memory access pattern degrades faster than the other two.

6. How Python's Interpreter, Data Structures, and Memory Model Shaped These Results

6.1 The interpreter

CPython compiles source to bytecode and executes it in a dispatch loop. Every operation the sorts perform — a comparison, an index, a loop increment — is one or more bytecode instructions, each requiring a dispatch, a stack operation, and dynamic type resolution. A single $a[i] < \text{pivot}$ costs several bytecodes plus a call into `PyObject_RichCompare`, which must determine at runtime that both operands are integers before any comparing happens. In C, that entire expression is one machine instruction on values already in registers.

The cost was measured directly rather than assumed. The C implementation in `csort.c` uses the identical algorithm as `sorts.py` — same median-of-three pivot, same Hoare partition, same tail-call elimination — so the algorithm is held constant and only the language varies:

<code>quick_sort</code> (Python, interpreted)	82.33 ms	
<code>c_quicksort</code> (C, compiled -O2)	5.29 ms	15.6x faster

A $15.6\times$ difference with the algorithm held fixed is the interpreter tax, stated as a number.

The $10\times$ gap to `list.sort` in Table 1 is the same effect measured a second way.

Two further interpreter properties matter for the service being designed. First, CPython 3.13 still enforces the **Global Interpreter Lock**, so a pure-Python sort cannot be parallelized across threads to recover the shortfall; on a 14-core machine, this workload uses one core. Second, CPython performs **no tail-call optimization**, which is why quick sort required explicit stack-depth management to run at all.

It is worth noting what the interpreter does well. Python 3.11 introduced a specializing adaptive interpreter that rewrites hot bytecode into type-specialized variants, and the results reflect it: 84 ms for 100,000 elements in pure Python would have been substantially slower on Python 3.9. The gap to compiled code has narrowed. It has not closed.

6.2 Data structures

A Python list is not an array of integers. It is a dynamically sized array of PyObject* pointers, each referring to a separately heap-allocated integer object. Two consequences follow directly from the measurements.

Every comparison costs an indirection. Comparing two elements means dereferencing two pointers to reach objects that may be anywhere on the heap, reading their type tags, dispatching to the integer comparison routine, and only then comparing values. A C array of int64 holds the values themselves, contiguously, so a comparison is a register operation on data the prefetcher has already fetched.

Cache locality is poor due to construction. The pointer array is contiguous, but the integer objects it points to are scattered. Sequential traversal of a Python list is sequential only in the pointers; the actual values arrive through effectively random access. This compounds with

algorithm choice, which is why heap sort — already the worst of the three for locality — degrades fastest as n grows (Table 4).

A further structural detail explains why merge sort's memory cost stayed low. Slicing a list copies *pointers*, not the integer objects. Merge sort allocates $O(n \log n)$ cumulative pointer storage but never duplicates a single integer, so its peak was 1.65 MB rather than the several hundred megabytes a value-copying language would need. Python's boxing costs time on every comparison and then refunds part of that cost during copying.

6.3 The memory model

The measured 3.82 MB for 100,000 integers decomposes exactly as the object model predicts:

Representation	Bytes per element	100,000 elements
Python list of distinct ints	8 (pointer) + 28 (int object) = 36	3.43 MB analytic
...as actually measured	includes list over-allocation	3.82 MB measured
Values 0–99 only (few_unique)	8 (pointer only)	0.77 MB measured
Packed int64 buffer (NumPy / C)	8	0.76 MB

Three things in that table are worth drawing out.

A Python integer costs 28 bytes. An int object carries a reference count, a type pointer, a digit count, and its value. The value itself is a small fraction of what is stored. Against a packed 64-bit buffer, Python's representation costs $4.5\times$ more memory for the same numbers.

Small-integer caching is visible in the data. The `few_unique` distribution — 100,000 values drawn from 0–99 — measured 0.77 MB rather than 3.82 MB. CPython pre-allocates and shares integer objects from -5 to 256, so that list contains 100,000 pointers to at most 100 shared objects. This is not an optimization the program requested; it is the interpreter's object model becoming visible under measurement, and it means Python's memory cost for integer data depends on the data's *values*, not only its size.

Reference counting keeps peak memory flat. Intermediate lists in merge sort are freed the moment their refcount drops to zero rather than at some later collection cycle, so cumulative allocation of $O(n \log n)$ never becomes a peak of $O(n \log n)$. Deterministic, immediate reclamation is why merge sort's peak stayed at 1.65 MB.

6.4 Reading the 50 MB requirement honestly

Peak memory has more than one defensible definition, and R2's verdict depends on which is meant. All three were measured:

Interpretation	Measured	vs 50 MB
Bytes the sort itself allocates (tracemalloc)	0.00 MB in place; 1.65 MB merge sort	passes, ~30× headroom
Sort allocation plus the input data	3.82 – 5.47 MB	passes, ~9–13× headroom
Resident set size of the whole process	21.6 MB with data; 34.8 MB with NumPy loaded	passes, ~1.4× headroom

Process memory grows as follows:

bare CPython interpreter, nothing imported	15.83 MB
+ list of 100,000 distinct ints	21.61 MB
+ merge_sort output list	24.34 MB
+ import numpy	34.75 MB

R2 is met under every reading, but the composition of that number deserves attention.

The 100,000 integers the service actually cares about occupy under 4 MB. The bare interpreter occupies 15.8 MB — **32% of the entire budget consumed before a single data element exists**, rising to 70% once NumPy is imported. Python passes this requirement, but it passes it with fixed overhead that no algorithm choice can reduce, and the remaining headroom must still

accommodate the rest of an analytics service. If the 50 MB ceiling applies per worker process in a multi-process deployment, that overhead is paid once per worker.

One limitation of the instrument should be recorded: tracemalloc only observes allocations made through CPython's own allocator. NumPy and ctypes buffers come from malloc directly and are invisible to it, reporting as 0.00 MB. Those rows are therefore reported using buffer size and process RSS instead. A harness that did not account for this would confidently report that NumPy uses no memory.

7. Mitigations

R2 and R3 are met and need no mitigation. R1 is met only on fast cores, so the mitigations below target it. Rather than recommend options on reputation, each was benchmarked on this machine through the same harness — which doubles as evidence that the harness is genuinely reusable, since it measured NumPy arrays and raw C buffers without modification.

Table 5 — Measured mitigations, 100,000 random integers. Every option passes the 400 ms budget on both tiers.

Option	P-core	E-core	Speedup	Assessment
quick_sort (pure Python)	82.3 ms	347.6 ms	1.0×	Baseline — the problem

list.sort (C Timsort)	8.3 ms	22.1 ms	9.9×	Free; one-line change
NumPy sort, data already ndarray	1.8 ms	5.0 ms	45.4×	Best result overall
NumPy sort, list round trip	4.5 ms	15.7 ms	18.4×	Still comfortably passes
C quicksort via ctypes, buffer	5.3 ms	8.2 ms	15.6×	Same algorithm, compiled
C quicksort via ctypes, round trip	16.0 ms	57.4 ms	5.2×	Marshalling dominates
libc qsort via ctypes, buffer	5.5 ms	12.1 ms	15.0×	Reference point

7.1 The distinction that matters: sort-only versus round trip

Each native option was measured twice, and the gap between the two readings is the real engineering decision:

- Sort only — the data already lives in a native buffer. This is the number achievable if the surrounding pipeline is rewritten to keep data out of Python objects.
- Round trip — the data arrives as a Python list and must be converted in and back out. This is the number achievable by swapping in a fast sort and changing nothing else.

For NumPy, conversion accounts for **57% of the round-trip time** — 1.23 ms to build the array and 1.18 ms to convert back, against 1.83 ms of actual sorting. The ctypes path is worse: 5.3 ms of sorting becomes 16.0 ms once marshalling is included, cutting the speedup from 15.6× to 5.2×. The lesson generalizes beyond this assignment. The benefit of a compiled kernel is bounded by how often data crosses the Python boundary, so these libraries pay off when data stays in native form across many operations, and disappoint when they are called once per Python object.

7.2 Recommendations, in order of cost

1. Use `list.sort`. Effort: one line. A 9.9× speedup that passes on both hardware tiers, with no dependency, no build step, and no conversion cost. Timsort is also adaptive and stable, so near-sorted analytics data sorts in a fraction of a millisecond (Table 3). For sorting specifically, the other options are only worth discussing because the activity requires implementing the algorithms by hand.

2. Adopt NumPy for the data path. Effort: moderate. $45\times$ faster sorting, $4.5\times$ less memory, and it addresses R1 and the memory composition problem in section 6.4 simultaneously. The condition for success is architectural: ingest into ndarray once and keep it there. An analytics service that filters, aggregates, and sorts is exactly the workload where that holds, so this is the recommendation for the service as designed.

3. Cython or a C extension. Effort: high. The ctypes measurements bound what this would deliver: roughly $15\times$ on the sort itself, less after marshalling. Cython would beat ctypes at the boundary by using the C API directly rather than converting through Python objects, but it adds a compiler toolchain, platform-specific build artifacts, and a debugging surface that spans two languages. Justified only for a hot path that NumPy cannot express — not for sorting, which NumPy already does better.

4. PyPy. Effort: low to try, high to commit. PyPy was not installed on the test machine, so no measurement is offered and none is claimed. Its JIT typically delivers large gains on exactly this profile — long-running numeric loops in pure Python — and it would likely be the strongest option that requires no code changes at all. The tradeoff is ecosystem: C-extension compatibility remains imperfect, and a service that later adopts NumPy or pandas may find PyPy the more constrained runtime. Worth an afternoon's evaluation before committing to option 3.

5. Change languages. Effort: highest. The C measurement is the honest benchmark for this: $15.6\times$ on the same algorithm. Go or Rust would land in similar territory with far better memory safety. This is justified only if profiling shows the service is dominated by element-wise

computation that resists vectorization. The evidence here does not support that — a $9.9\times$ win is available from a one-line change, and $45\times$ from a library already installed.

7.3 What was ruled out

- Threading. The GIL prevents pure-Python CPU-bound work from scaling across threads. Multiprocessing could parallelize a merge across workers, but it pays inter-process serialization and duplicates the 15.8 MB interpreter per worker — pressing directly on R2.
- Micro-optimizing the Python. Hoisting method lookups and minimizing attribute access is already done in these implementations. Such tuning yields tens of percent; the shortfall on efficiency cores requires a factor of two or more.
- A better algorithm. Table 4 shows all three already achieve $O(n \log n)$. The problem is the constant factor, which algorithm selection cannot address.

8. Conclusion

Python satisfies two of the three requirements outright and the third only conditionally. R2 is met with $9\text{--}13\times$ headroom on the data itself, though the interpreter's fixed 15.8 MB consumes a third of the budget before any data exists. R3 is met by `benchmark.py`. R1 is met on performance cores with $4.8\times$ headroom and fails on efficiency cores, where the best pure-Python sort exceeded the budget on half of its runs.

The correct conclusion is not that Python is too slow for this service. It is that **pure-Python element-wise loops** are too slow for this service, and Python the ecosystem

already contains the remedy. The same 400 ms budget that pure-Python quick sort misses on slow hardware is met with an $18\times$ margin by a built-in function, and a $45\times$ margin by a library installed in most analytics environments. Python's cost is paid per interpreted operation, so the engineering discipline the language demands is to keep the operation count low and delegate the inner loop to compiled code.

Recommended decision: adopt Python. Standardize on NumPy for the analytics data path, use `list.sort` wherever ordinary lists are sorted, and adopt a standing rule that any operation inside a latency budget must be delegated to compiled code rather than expressed as a Python loop. Re-run the harness in this report against the actual deployment hardware before the budget is finalized — the central finding of this study is that the answer to R1 changes with the machine, and it should therefore be measured on the machine that will run it.

9. Acknowledgment of Generative AI Use

Per the course AI usage guidelines: Anthropic's Claude (Opus 5) was used to help edit this report (Anthropic, 2026). The author reviewed and approved the analysis and final recommendation.

Anthropic. (2026). Claude (Opus 5) [Large language model]. <https://claude.ai>

Appendix A — Submitted Files

File	Contents
sorts.py	Quick sort, merge sort, and heap sort in pure Python

benchmark.py	The reusable benchmarking harness (rubric criterion 2)
run_benchmarks.py	Driver: judges all three algorithms against R1–R3
mitigations.py	Measures NumPy, ctypes/C, and list.sort mitigations
rss_breakdown.py	Process memory against the 50 MB budget
csort.c	C quicksort, algorithmically identical to sorts.quick_sort
results/*.csv, *.json	Raw measurements for both hardware tiers
README.md	How to reproduce every figure in this report

Appendix B — Reproducing These Results

```
python3 run_benchmarks.py --tier p-core    # full suite, performance cores
taskpolicy -b python3 run_benchmarks.py --tier e-core # efficiency cores (macOS)
python3 mitigations.py --tier p-core       # measured mitigation options
python3 rss_breakdown.py                  # process memory vs the 50 MB budget
```

The random seed is fixed at 660, so every run sorts identical data. Absolute timings will differ on other hardware; the ratios between implementations should not.